🧠 Pneumonia Detection using Transfer Learning (VGG16)

🎯 Objective

The objective of this project is to implement a deep learning model based on the VGG16 transfer learning architecture to classify chest X-ray images as **Normal** or **Pneumonia**.

The model is developed using TensorFlow/Keras and evaluated using standard machine learning metrics such as accuracy, confusion matrix, precision, recall, and F1-score.

🎓 Academic Context

This project was completed as part of a **M.Tech academic at BITS Pilani**.
It is intended to demonstrate concepts in deep learning and computer vision within an academic setting.

⚠️ Disclaimer

- This project is created for **educational purposes only**
- It is **not intended for clinical use, medical diagnosis, or decision-making**
- The results are experimental and should **not be considered medical advice**
- This work represents an **independent academic implementation** and is not affiliated with or endorsed by any medical institution

---

📚 Paper Reference

This project is inspired by existing research on deep learning-based pneumonia detection using convolutional neural networks and transfer learning techniques.

In particular, it draws conceptual guidance from approaches that utilize pretrained architectures such as VGG16 for medical image classification tasks.

⚠️ **Note:**
This implementation is an independent and simplified learning exercise. It does not reproduce or replicate the referenced research paper, and no content, code, or proprietary methodology has been copied. The work is intended solely for educational purposes.

In [1]:
```
import json

kaggle_dict = {
    "username": "<user_name>",
    "key": "<key>"
}

with open("kaggle.json", "w") as f:
    json.dump(kaggle_dict, f)

print("kaggle.json created successfully!")
```

```
kaggle.json created successfully!
```

⚙️ Setup and Data Acquisition

This section outlines the steps for environment setup and dataset retrieval:

1. **Kaggle API Configuration**:
   Users must generate their own Kaggle API credentials and place the `kaggle.json` file locally to authenticate dataset downloads.
   ⚠️ Do not share or upload your API credentials publicly.

2. **Environment Preparation**:
   Installs the Kaggle CLI and configures the local environment, including setting appropriate file permissions for secure access to the API key.

3. **Dataset Download**:
   Downloads the *Chest X-Ray Images (Pneumonia)* dataset directly from Kaggle and extracts it for training.

⚠️ Important Notes

- This project does **not include or redistribute the dataset**
- Users must download the dataset directly from Kaggle and comply with its **license and usage terms**
- All data used is assumed to be **publicly available and de-identified**

In [2]:
```
!pip install -q kaggle
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

In [3]:
```
!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia
!unzip chest-xray-pneumonia.zip
```

```
Streaming output truncated to the last 5000 lines.
  inflating: chest_xray/train/NORMAL/IM-0435-0001-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0435-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0437-0001-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0437-0001-0002.jpeg
  inflating: chest_xray/train/NORMAL/IM-0437-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0438-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0439-0001-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0439-0001-0002.jpeg
  inflating: chest_xray/train/NORMAL/IM-0439-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0440-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0441-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0442-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0444-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0445-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0446-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0447-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0448-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0449-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0450-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0451-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0452-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0453-0001-0002.jpeg
  inflating: chest_xray/train/NORMAL/IM-0453-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0455-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0456-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0457-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0458-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0459-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0460-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0461-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0463-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0464-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0465-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0466-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0467-0001-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0467-0001-0002.jpeg
  inflating: chest_xray/train/NORMAL/IM-0467-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0469-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0471-0001.jpeg
  inflating: chest_xray/train/NORMAL/IM-0472-0001.jpeg
```

📂 Dataset

Chest X-Ray Pneumonia Dataset
Source: Kaggle
https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia

This project uses a publicly available dataset provided on Kaggle for educational and research purposes.

⚠️ **Note:**

- This repository does **not include the dataset**
- Users must download the dataset directly from Kaggle
- Please ensure compliance with the dataset's **license and terms of use** before using it
- The dataset is assumed to contain **de-identified medical images**

In [4]:
```
import os

base_path = "chest_xray"

print("Train folders:", os.listdir(os.path.join(base_path, "train")))
print("Validation folders:", os.listdir(os.path.join(base_path, "val")))
print("Test folders:", os.listdir(os.path.join(base_path, "test")))
```

```
Train folders: ['PNEUMONIA', 'NORMAL']
Validation folders: ['PNEUMONIA', 'NORMAL']
Test folders: ['PNEUMONIA', 'NORMAL']
```

📊 Dataset Description

The dataset used in this project is the **Chest X-Ray Pneumonia dataset**, publicly available on Kaggle.

- Total images: 5,840
- Classes: Normal and Pneumonia
- Folder structure: train, validation, test
- Images resized to 224×224 for compatibility with VGG16

The dataset is slightly imbalanced, with more Pneumonia images than Normal images.

⚠️ **Note:**

- This project does **not own or distribute the dataset**
- The dataset is accessed from Kaggle and used under its **respective license**
- Users must ensure compliance with the dataset's **terms of use** before using it
- The dataset is assumed to contain **de-identified medical images** and is used strictly for educational purposes

In [16]:
```
import os

normal_count = len(os.listdir("chest_xray/train/NORMAL"))
pneumonia_count = len(os.listdir("chest_xray/train/PNEUMONIA"))

print("Normal images:", normal_count)
print("Pneumonia images:", pneumonia_count)
```

```
Normal images: 1341
Pneumonia images: 3875
```

🏗️ Model Architecture

This project utilizes the **VGG16 pretrained model** with ImageNet weights as a feature extractor for image classification.

The original top classification layers were removed and replaced with custom layers:

- **GlobalAveragePooling2D**: Reduces the spatial dimensions of extracted features
- **Dense layer (512 units, ReLU activation)**: Learns high-level representations from the extracted features
- **Dropout (0.5)**: Helps reduce overfitting by randomly deactivating neurons during training
- **Output layer (Sigmoid activation)**: Produces a probability score for binary classification (Normal vs. Pneumonia)

The convolutional base was frozen during training to leverage pretrained feature representations and improve training efficiency.

⚠️ **Note:**

- This architecture is implemented for **educational and experimental purposes only**
- The model design is a simplified adaptation of standard transfer learning practices and may not reflect clinically validated systems
- The outputs of this model should **not be used for medical diagnosis or decision-making**

📊 EDA Findings

- **Visual Difference**:
  In this dataset, images labeled as Pneumonia often appear to have increased opacity or "cloudiness" in lung regions, while Normal images tend to show clearer lung fields.
  ⚠️ This observation is based on dataset visualization and should not be interpreted as a medical or diagnostic conclusion.

- **Imbalance**:
  The training set contains significantly more Pneumonia cases (~ 3800) than Normal cases (~ 1300).
  To address this imbalance, **data augmentation** techniques are applied during training to improve model generalization and reduce bias toward the majority class.

⚠️ **Note:**

- Observations are derived from exploratory data analysis for **educational purposes only**
- Visual patterns described here do **not represent clinical diagnostic criteria**

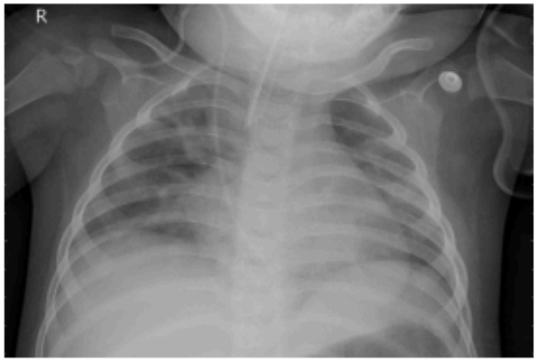
In [17]:
```
import matplotlib.pyplot as plt
import cv2
import os
import random

def plot_samples(class_name):
    path = f'chest_xray/train/{class_name}'
    imgs = random.sample(os.listdir(path), 3)
    plt.figure(figsize=(12, 4))
    for i, img_name in enumerate(imgs):
        plt.subplot(1, 3, i+1)
        img = cv2.imread(os.path.join(path, img_name))
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
        plt.title(f'{class_name} Sample')
        plt.axis('off')
    plt.show()

print('Displaying Normal Samples:')
plot_samples('NORMAL')
print('Displaying Pneumonia Samples:')
plot_samples('PNEUMONIA')
```

```
Displaying Normal Samples:
```

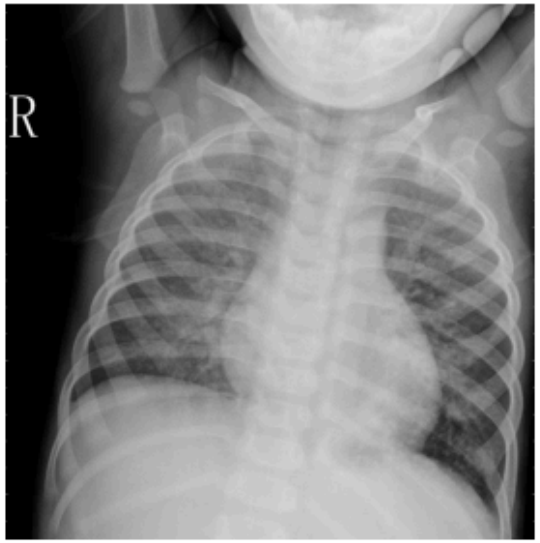
```
Displaying Pneumonia Samples:
```

PNEUMONIA Sample



PNEUMONIA Sample

PNEUMONIA Sample



📊 EDA Findings

- **Visual Difference**:
  Within this dataset, images labeled as Pneumonia may exhibit higher opacity or "cloudiness" in lung regions, whereas Normal images generally appear to have clearer lung fields.
  ⚠️ These observations are based solely on dataset visualization and do not constitute medical or diagnostic conclusions.

- **Imbalance**:
  The training set contains significantly more Pneumonia cases (~ 3800) than Normal cases (~ 1300).
  To address this imbalance, **data augmentation** techniques are applied during training to improve model generalization and reduce bias toward the majority class.

⚠️ **Note:**

- All observations are derived from exploratory data analysis conducted for **educational and research purposes only**
- The visual characteristics described do **not represent clinical diagnostic standards or medical guidance**

⚙️ Training Strategy

To facilitate effective learning and reduce overfitting, the following training configuration was used:

- **Optimizer: Adam**:
  An adaptive optimization algorithm that adjusts learning rates for individual parameters during training
- **Loss Function: Binary Crossentropy**:
  Commonly used loss function for binary classification tasks
- **Metrics: Accuracy**:
  Used to monitor the proportion of correctly classified samples
- **Callbacks used**:
  - **EarlyStopping**: Stops training when validation loss does not improve for 3 consecutive epochs
  - **ModelCheckpoint**: Saves model weights corresponding to the best validation performance
  - **ReduceLROnPlateau**: Reduces the learning rate when validation performance plateaus
- **Epochs: 10**:
  The model was trained for up to 10 epochs over the dataset

⚠️ **Note:**

- The training configuration is selected for **educational and experimental purposes**
- Hyperparameters and strategies may not be optimized for real-world or clinical applications
- Results may vary depending on dataset splits, preprocessing, and training conditions

In [8]:
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

img_size = 224
batch_size = 32

# Training data generator (with augmentation + validation split)
```

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.2   # 20% for validation
)

# Training subset (80%)
train_generator = train_datagen.flow_from_directory(
    "chest_xray/train",
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='binary',
    subset='training'
)

# Validation subset (20%)
val_generator = train_datagen.flow_from_directory(
    "chest_xray/train",
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='binary',
    subset='validation'
)

# Test generator (NO augmentation)
test_datagen = ImageDataGenerator(rescale=1./255)

test_generator = test_datagen.flow_from_directory(
    "chest_xray/test",
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=False
)

# Print counts clearly
print("Training samples:", train_generator.samples)
```

```
print("Validation samples:", val_generator.samples)
print("Test samples:", test_generator.samples)
```

```
Found 4173 images belonging to 2 classes.
Found 1043 images belonging to 2 classes.
Found 624 images belonging to 2 classes.
Training samples: 4173
Validation samples: 1043
Test samples: 624
```

In [9]:

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout

# Load pretrained VGG16
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze convolution layers
for layer in base_model.layers:
    layer.trainable = False

# Add custom classification head
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=output)

model.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kerne
ls_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0

global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 512)	262,656
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 1)	513

Total params: 14,977,857 (57.14 MB)

Trainable params: 263,169 (1.00 MB)

Non-trainable params: 14,714,688 (56.13 MB)

In [10]:
```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

In [11]:
```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

checkpoint = ModelCheckpoint(
    "best_model.h5",
    monitor='val_accuracy',
    save_best_only=True
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=2
)
```

In [12]:
```
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=[early_stop, checkpoint, reduce_lr]
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDatase
t` class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignored.
  self._warn_if_super_not_called()
Epoch 1/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 0s 682ms/step - accuracy: 0.7876 - loss: 0.4605
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.
save_model(model, 'my_model.keras')`.
131/131 ━━━━━━━━━━━━━━━━━━━━ 137s 937ms/step - accuracy: 0.7880 - loss: 0.4596 - val_accuracy: 0.8926 - val_loss: 0.2537 - lear
ning_rate: 0.0010
Epoch 2/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 0s 638ms/step - accuracy: 0.9029 - loss: 0.2379
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.
save_model(model, 'my_model.keras')`.
131/131 ━━━━━━━━━━━━━━━━━━━━ 105s 804ms/step - accuracy: 0.9029 - loss: 0.2379 - val_accuracy: 0.9070 - val_loss: 0.2183 - lear
ning_rate: 0.0010
Epoch 3/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 0s 638ms/step - accuracy: 0.9181 - loss: 0.2139
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.
save_model(model, 'my_model.keras')`.
131/131 ━━━━━━━━━━━━━━━━━━━━ 105s 800ms/step - accuracy: 0.9181 - loss: 0.2139 - val_accuracy: 0.9089 - val_loss: 0.1977 - lear
ning_rate: 0.0010
Epoch 4/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 0s 632ms/step - accuracy: 0.9162 - loss: 0.2143
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.
save_model(model, 'my_model.keras')`.
```

```
131/131 ━━━━━━━━━━━━━━━━━━━━ 104s 793ms/step - accuracy: 0.9162 - loss: 0.2142 - val_accuracy: 0.9329 - val_loss: 0.1724 - lear
ning_rate: 0.0010
Epoch 5/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 105s 800ms/step - accuracy: 0.9326 - loss: 0.1805 - val_accuracy: 0.9108 - val_loss: 0.1823 - lear
ning_rate: 0.0010
Epoch 6/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 104s 795ms/step - accuracy: 0.9275 - loss: 0.1847 - val_accuracy: 0.9310 - val_loss: 0.1656 - lear
ning_rate: 0.0010
Epoch 7/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 104s 795ms/step - accuracy: 0.9253 - loss: 0.1861 - val_accuracy: 0.9281 - val_loss: 0.1791 - lear
ning_rate: 0.0010
Epoch 8/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 0s 632ms/step - accuracy: 0.9309 - loss: 0.1790
```

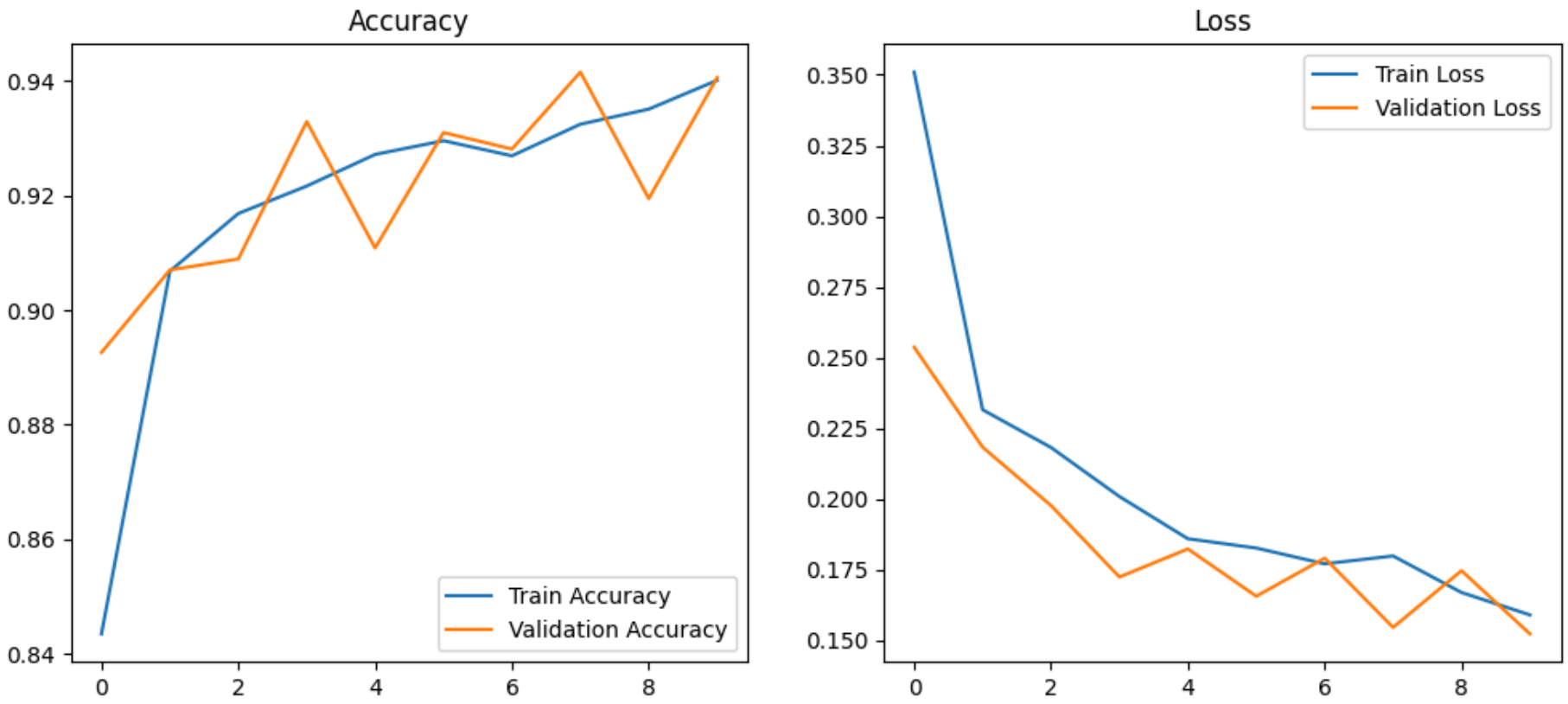
```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.
save_model(model, 'my_model.keras')`.
```

```
131/131 ━━━━━━━━━━━━━━━━━━━━ 141s 786ms/step - accuracy: 0.9309 - loss: 0.1790 - val_accuracy: 0.9415 - val_loss: 0.1546 - lear
ning_rate: 0.0010
Epoch 9/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 104s 794ms/step - accuracy: 0.9333 - loss: 0.1643 - val_accuracy: 0.9195 - val_loss: 0.1746 - lear
ning_rate: 0.0010
Epoch 10/10
131/131 ━━━━━━━━━━━━━━━━━━━━ 105s 801ms/step - accuracy: 0.9398 - loss: 0.1690 - val_accuracy: 0.9406 - val_loss: 0.1523 - lear
ning_rate: 0.0010
```

In [13]:

```python
import matplotlib.pyplot as plt

# Accuracy Plot
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Accuracy')
```

```
# Loss Plot
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss')

plt.show()
```



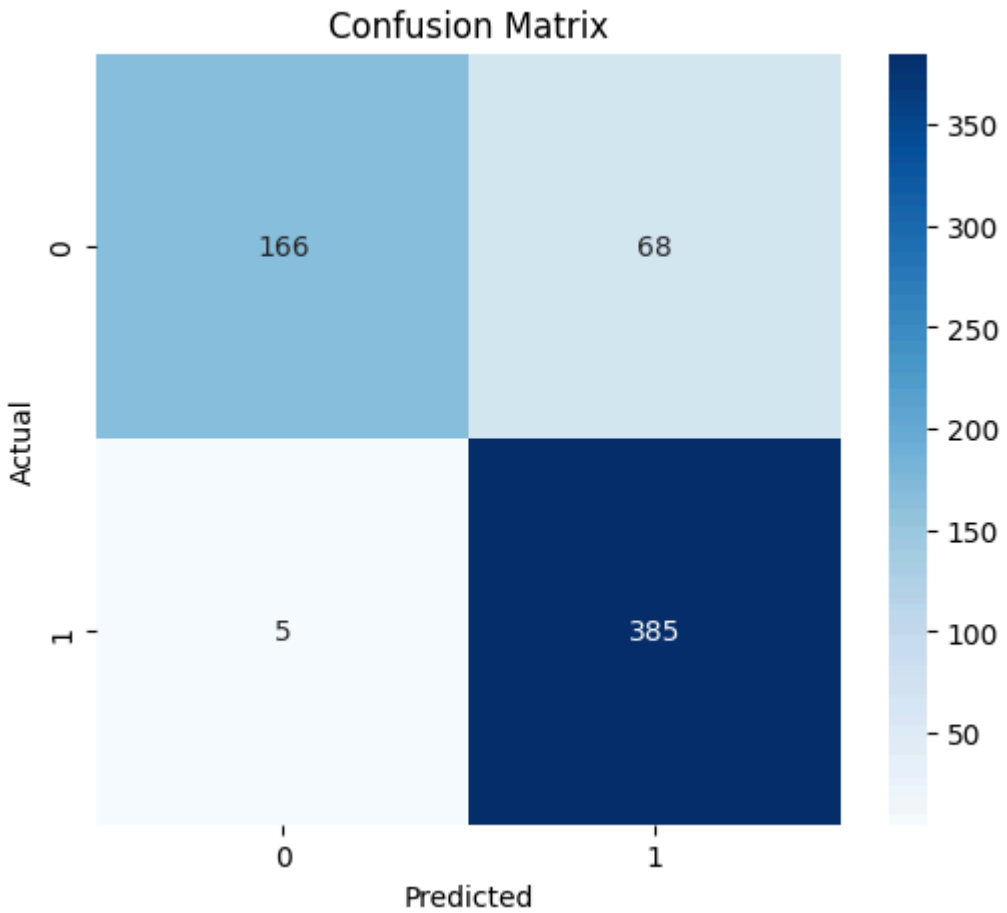
In [14]:
```
test_loss, test_accuracy = model.evaluate(test_generator)
print("Test Accuracy:", test_accuracy)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDatase
t` class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignored.
  self._warn_if_super_not_called()
20/20 ━━━━━━━━━━━━━━━━━━━━ 16s 778ms/step - accuracy: 0.8192 - loss: 0.5155
Test Accuracy: 0.8830128312110901
```

In [15]:

```python
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Predictions
predictions = model.predict(test_generator)
y_pred = (predictions > 0.5).astype(int).reshape(-1)
y_true = test_generator.classes

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

print(classification_report(y_true, y_pred))
```

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 10s 301ms/step
```




```
              precision    recall  f1-score   support

           0       0.97      0.71      0.82       234
           1       0.85      0.99      0.91       390

    accuracy                           0.88       624
   macro avg       0.91      0.85      0.87       624
weighted avg       0.90      0.88      0.88       624
```

🧪 Results and Discussion

The VGG16-based transfer learning model achieved a **validation accuracy of 94.06%** and a **test accuracy of 88.30%**.

The confusion matrix indicates strong classification performance on this dataset, with a **recall of 99% for the Pneumonia class**, meaning the model correctly identified most pneumonia-labeled images in the test set.

Only **5 false negatives** were observed in this experimental setup, reflecting high sensitivity within the dataset used.

The model shows a slightly higher number of false positives, meaning it occasionally predicts pneumonia for normal cases. This trade-off is common in binary classification tasks and depends on model thresholds and dataset characteristics.

Overall, the model demonstrates promising performance on the given dataset and highlights the potential of transfer learning techniques such as VGG16 for medical image classification tasks.

⚠️ **Important Note:**

- These results are based on a **specific dataset and experimental setup**
- Performance metrics may not generalize to real-world or clinical environments
- This model is **not validated for medical use** and should not be used for diagnosis or healthcare decision-making
- Any references to healthcare contexts are for **conceptual understanding only**, not practical deployment

🔓 Legal, Ethical, and Usage Notice

📌 Educational Purpose

This project is developed strictly for educational and research purposes as part of an academic exercise. It is intended to demonstrate concepts in deep learning, computer vision, and transfer learning.

⚠️ Medical Disclaimer

This project is **not a medical device** and has **not undergone clinical validation or regulatory approval**.
It must **not be used for medical diagnosis, treatment, or decision-making** in any real-world healthcare setting.

Any outputs generated by this model are purely experimental and are intended only for academic understanding.

📊 Dataset Usage and License

This project utilizes the *Chest X-Ray Pneumonia Dataset* available on Kaggle:
👉 https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia

- The dataset is **not included or redistributed** in this repository
- Users must download the dataset directly from Kaggle
- Use of the dataset is subject to its **original license and terms of use**
- It is the **user's responsibility** to ensure compliance with those terms before use

The dataset is provided by its original authors and is **represented as containing de-identified medical images**.

🔐 Data Privacy and Ethics

No personally identifiable information (PII) is intentionally used in this project.
All data is sourced from publicly available datasets intended for research and educational use.

💻 Code License

Unless otherwise stated, the code in this project is released under the **MIT License**, which permits reuse, modification, and distribution with proper attribution.

🔑 API Key and Security Notice

If using the Kaggle API:

- Do **not share your `kaggle.json` file or API credentials publicly**
- Keep your API keys secure to prevent unauthorized access

📉 Model Limitations

- Model performance is based on a **specific dataset and experimental setup**
- Results may not generalize to real-world or clinical environments
- The model may produce **false positives or false negatives**
- No guarantee of accuracy, reliability, or suitability for any purpose is provided

---

⚖️ Liability Disclaimer

The authors assume **no responsibility or liability** for:

- Any misuse of this project
- Any decisions made based on its outputs
- Any direct or indirect consequences arising from its use

Use this project **at your own risk**.

📚 Research Acknowledgment

This work is inspired by existing research in deep learning for medical image classification.
It is an independent implementation created for learning purposes and does not reproduce proprietary methods or content.

In [ ]:

```

```